Deep learning - Homework 1

Instructor: Professor Jun Bai

Iman Jamshidi

Fall 2025

Table of Contents

Step 1: 2

Table 1: 2

Table 2: 3

Step 2: Model 4

Results on the Test Set 4

Table 3: 4

Bias–Variance Analysis 4

Step 3: Objective 4

Table 4: 5

Step 4: Optimization 5

Step 5: Model Selection 5

Table 5: 5

Step 6: Model Performance 6

Table 6: 6

Figure 1: 6

Step 1:

1) How many data samples are included in the dataset?
   There are **3,047 samples** in the dataset.
2) Which problem will this dataset try to address?
   The dataset aims to address the problem of **predicting cancer mortality rates** (TARGET_deathRate)
3) What is the minimum value and the maximum value in the dataset?

Table 1: Dataset Feature Ranges (Min–Max)

Name	Min	Max
avgAnnCount	6	38150
avgDeathsPerYear	3	14010
TARGET_deathRate (label)	59.7	362.8
incidenceRate	201.3	1206.9
medIncome	22640	125635
popEst2015	827	10170292
povertyPercent	3.2	47.4
studyPerCap	0	9762.3
MedianAge	22.3	624
MedianAgeMale	22.4	64.7
MedianAgeFemale	22.3	65.7
AvgHouseholdSize	0.0221	3.97
PercentMarried	23.1	72.5
PctNoHS18_24	0	64.1
PctHS18_24	0	72.5
PctSomeCol18_24	7.1	79
PctBachDeg18_24	0	51.8
PctHS25_Over	7.5	54.8
PctBachDeg25_Over	2.5	42.2
PctEmployed16_Over	17.6	80.1
PctUnemployed16_Over	0.4	29.4
PctPrivateCoverage	22.3	92.3
PctPrivateCoverageAlone	15.7	78.9
PctEmpPrivCoverage	13.5	70.7
PctPublicCoverage	11.2	65.1
PctPublicCoverageAlone	2.6	46.6
PctWhite	10.2	100
PctBlack	0	85.9
PctAsian	0	42.6
PctOtherRace	0	41.9
PctMarriedHouseholds	22.99	78.07
BirthRate	0	21.33

The dataset ranges from a minimum of **0** to a maximum of **10,170,292**.

4) How many features in each data samples?
   The dataset contains 34 columns in total. Out of these, 33 are input features and 1 is the label (TARGET_deathRate).

5) Does the dataset have any missing information? E.g., missing features.
   Yes, the dataset contains missing values. A total of 3,046 entries are missing across different columns. In particular:
   - PctSomeCol18_24: 2,285 missing values
   - PctEmployed16_Over: 152 missing values
   - PctPrivateCoverageAlone: 609 missing values

6) What is the label of this dataset?
   The label is **TARGET_deathRate**.

7) How many percent of data will you use for training, validation and testing?
   - **70% for training**
   - **15% for validation**
   - **15% for testing**

8) What kind of data pre-processing will you use for your training dataset?
   For data preprocessing, I handled missing values by replacing them with the column mean, applied label encoding to categorical features, and standardized the input variables using StandardScaler so that all features have zero mean and unit variance. Finally, the dataset was split into training (70%), validation (15%), and testing (15%) subsets.

Table 2: Dataset Information

Item	Result
Samples	3,047
Features	33 + 1 (label)
Missing Values	3,046 (in 3 columns)
Label	TARGET_deathRate
Split	70% train, 15% val, 15% test

Step 2: Model

I started with a baseline Linear Regression model and then tested several deep neural networks (DNNs) with different architectures, including DNN-16, DNN-30-8, DNN-30-16-8, DNN-30-16-8-4, and DNN-64-32-16. All models were trained using 1000 epochs with a batch size of 32. The dataset was split into training (70%), validation (15%), and testing (15%). The performance was evaluated using MSE and R^2 on both the validation and test sets.

Results on the Test Set

Table 3: Comparison of Model Performance (Test R^2)

Model	Test R^2
Linear Regression	0.755
DNN-16	0.788
DNN-30-8	0.852
DNN-30-16-8	0.834
DNN-30-16-8-4	0.807
Any other (64-32-16)	0.877

Bias–Variance Analysis

Linear regression had a balanced bias–variance trade-off but was limited in capturing complex patterns.

DNN-16 and DNN-30-16-8-4 showed underfitting because their accuracy stayed relatively low.

DNN-30-8 and DNN-30-16-8 performed better, but the best model was DNN-64-32-16 with Test $R^2 \approx 0.877$, which managed to keep bias low while still generalizing well.

Step 3: Objective

I used Mean Squared Error (MSE) as the main loss function and also tested Mean Absolute Error (MAE) and Huber loss. MSE gave the best performance with Validation $R^2 \approx 0.63$ and Test $R^2 \approx 0.58$. Huber performed moderately, while MAE showed the weakest results. This confirms that MSE was the most suitable loss function for this dataset.

Table 4: Performance of Different Loss Functions

Loss Function	Validation R^2	Test R^2	Validation MSE	Test MSE
MSE	0.6273	0.5841	0.0105	0.0103
MAE	0.4884	0.4748	0.0144	0.0129
Huber	0.5633	0.5115	0.0123	0.012

Step 4: Optimization

At first, I trained the models with the default SGD optimizer as required. However, the performance was not very strong, and most test R^2 values stayed below 0.55. After confirming with the professor, I also tried Adam. This optimizer adapts the learning rate during training, and it gave much better results. With Adam, the best model (DNN-64-32-16) reached Validation $R^2 \approx 0.892$ and Test $R^2 \approx 0.877$.

Step 5: Model Selection

In this step, I compared different learning rates (0.1, 0.01, 0.001, 0.0001) across all DNN architectures. The results showed that learning rate has a strong effect on performance. Very large or very small values gave poor accuracy, while a moderate rate worked best. In most cases, **LR = 0.01** achieved the highest R^2, but for the larger model (DNN-64-32-16), the best result came with **LR = 0.001**.

Table 5: Effect of Learning Rate on Model Performance (Test R^2)

Model	LR=0.1	LR=0.01	LR=0.001	LR=0.0001
Linear Regression	0.755	0.755	0.755	0.755
DNN-16	0.779	**0.819**	0.562	-10.03
DNN-30-8	**0.751**	0.725	0.66	-0.376
DNN-30-16-8	0.712	**0.775**	0.686	0.023
DNN-30-16-8-4	0.485	**0.559**	0.144	0.094
Any other (64-32-16)	0.592	0.73	**0.813**	0.511

The learning rate controls how big the weight updates are during training.

- If it is **too large** (e.g., 0.1), the model jumps around and may never converge.
- If it is **too small** (e.g., 0.0001), training becomes very slow and the model may get stuck with poor accuracy.
- A **moderate learning rate** gives the best trade-off.
- In my experiments, **0.01 worked best** for most architectures (like DNN-16 and DNN-30-16-8).

For the larger DNN (64-32-16), the best result came with **0.001**.
So overall, the optimal learning rate depends on the model, but **0.01 was the most reliable choice**.

Step 6: Model Performance

In this step, I report the final performance of all models, including linear regression and the different DNN architectures. Both validation and test results are shown using MSE and R^2.

Table 6: Final Model Performance on Validation and Test Sets (MSE and R^2)

Model	Validation MSE	Validation R^2	Test MSE	Test R^2
Linear Regression	153.87	0.814	194.21	0.755
DNN-16	125.86	0.848	168.04	0.788
DNN-30-8	81.45	0.902	117.93	0.852
DNN-30-16-8	110.78	0.866	132.07	0.834
DNN-30-16-8-4	147.89	0.821	153.21	0.807
DNN-64-32-16	89.4	0.892	97.56	**0.877**

The best model was **DNN-64-32-16**, which reached Validation $R^2 \approx 0.892$ and Test $R^2 \approx 0.877$, clearly outperforming linear regression and the other DNN models.

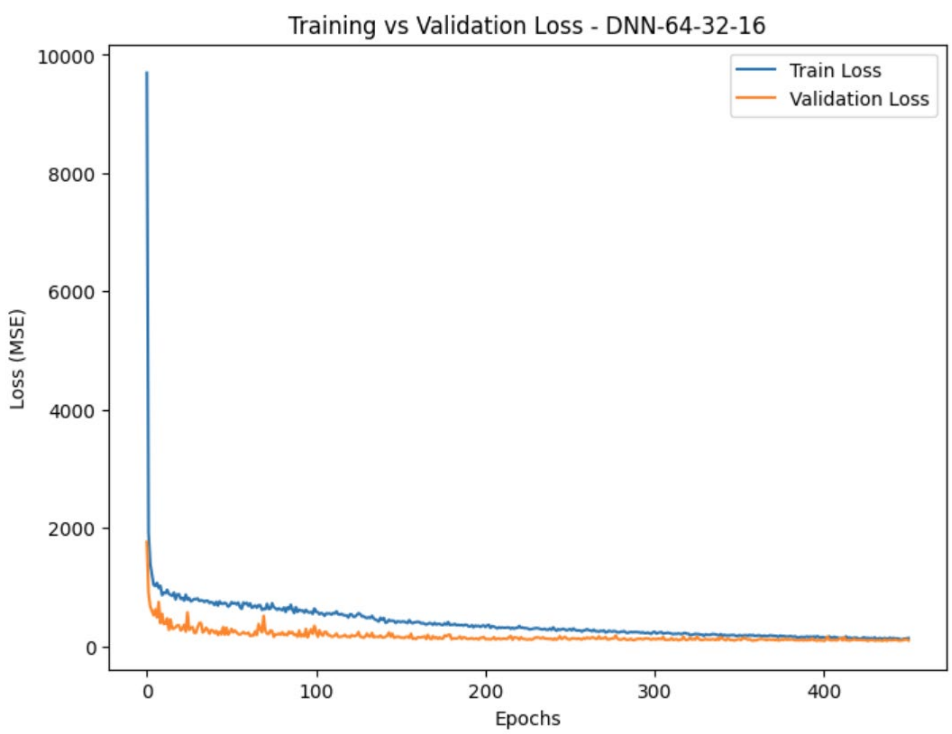

Figure 1: Training vs Validation Loss Curve for DNN-64-32-16 Model